44. Backups, Snapshots, and Restore Drills

44. Backups, Snapshots, and Restore Drills

Phase: Phase 8 — Packages, disks, storage
Estimated time: 1–2 days
Slug: `44-storage-backup`

Learning objectives

- Define RPO/RTO simply
- Prefer tested restores
- Know snapshot vs backup
- Encrypt backups and manage keys

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

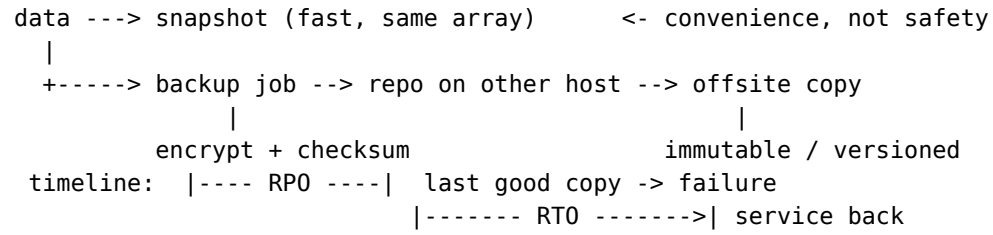
Why this matters

Everyone has backups until the day they need one. Disks fail, people run `rm -rf` in the wrong directory, ransomware encrypts both the server and the mounted share, and cloud accounts get deleted. The only question that matters afterwards is: how much data did we lose, and how long were we down? Those are your **RPO** and **RTO**, and they must be decided before the incident, tested in a drill, and written down.

RPO (recovery point objective) is the maximum data-loss window: the longest span of recent changes you accept losing. If you back up nightly, your RPO is about 24 hours, whatever the slide deck says. RTO is how long recovery may take. Untested backups are fiction.

Core concepts

Term	Meaning
RPO	max data-loss window, set by backup frequency
RTO	max acceptable time to restore service
Snapshot	point-in-time copy, usually on the same storage system
Backup	independent copy on separate media or location
Full / incremental	whole copy vs only changes since last run
Retention	how many restore points you keep and for how long
3-2-1	3 copies, 2 media types, 1 offsite



A snapshot on the same disk array dies with the array. A backup that you have never restored is a hope, not a control.

Worked examples

Example 1: a simple file backup with rsync and hard-link snapshots.

```
SRC=/srv/app/data
DST=/backup/app
TS=$(date +%Y%m%d-%H%M)
rsync -a --delete --link-dest="$DST/latest" "$SRC/" "$DST/$TS/"
ln -sfn "$DST/$TS" "$DST/latest"
```

Unchanged files are hard links, so each run looks like a full copy but costs only the changes.

Example 2: a tar archive with a checksum, ready to copy offsite.

```
tar -C /srv/app -czf /tmp/app-$(date +%F).tgz data
sha256sum /tmp/app-$(date +%F).tgz | tee /tmp/app-$(date +%F).tgz.sha256
```

Example 3: encrypt before it leaves the host.

```
gpg --symmetric --cipher-algo AES256 /tmp/app-2026-10-21.tgz
```

Store the passphrase or key separately from the backup. A backup you cannot decrypt is no backup.

Example 4: dedup backup with restic (concept-level).

```
restic -r /backup/restic init
restic -r /backup/restic backup /srv/app/data
restic -r /backup/restic snapshots
restic -r /backup/restic check
```

Example 5: a restore drill into a scratch directory, then compare.

```
mkdir -p /tmp/restore-test
tar -C /tmp/restore-test -xzf /tmp/app-2026-10-21.tgz
diff -r /srv/app/data /tmp/restore-test/data && echo RESTORE_OK
```

Mini-lab

1. Create `/tmp/lab/data` with ten small files.
2. Run Example 1 twice, modifying one file between runs. Use `ls -li` on a file in both snapshots to see shared inodes for unchanged files.

3. Create a tar archive plus a sha256 file. Corrupt a copy with `printf x >> copy.tgz` and show that `sha256sum -c` fails.
4. Delete the original data directory and restore from the backup.
5. Time the restore. Write down: this is my measured RTO for this dataset.
6. State your RPO given how often you ran the job. Write both numbers in a one-line runbook note.

Decision guide / failure modes

Situation	Choice
Need to roll back a risky change in minutes	snapshot (then still back up)
Protect against disk, host, or site loss	backup to another host plus offsite
Database in use	dump or snapshot with consistency, not raw copy of live files
Very low RPO	frequent incrementals, WAL archiving, replication (not a backup)
Ransomware concern	immutable or write-once copy, separate credentials

Failure modes: backups stored on the same disk; replication mistaken for backup (it replicates the deletion too); no checksum or `restic check`; keys lost; retention so short that corruption discovered on day 10 is already overwritten; and never timing a restore so RTO is a guess. Schedule a restore drill at least quarterly and record the result.

Common mistakes

- Only on-disk snapshots in same AZ
- No restore rehearsal
- Unencrypted sensitive backups

Exercises

Work in `exercises/44-storage-backup/`. Read that folder's `README.md` fully before coding.

```
cd exercises/44-storage-backup
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Open `exercise.sh` and implement `rpo_means`. It must print one lowercase token, with words joined by hyphens, that defines what RPO measures in the lesson's own words. No file access and no tools are required. After the test turns green, do the mini-lab restore drill and record two numbers in your notes: your measured restore time (RTO) and the gap between backup runs (your effective RPO). Finally explain in one sentence why a snapshot on the

same disk array does not satisfy a 3-2-1 plan. These notes are not graded but are the point of the module.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/44-storage-backup/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. RPO vs RTO?
2. Why restore drills?
3. 3-2-1 idea?

Further reading

- https://www.veeam.com/blog/321-backup-rule.html